In422 Real Time Information System
Real-Time Information System Project Report

Written by:

Lucas LAVIGNE

IPSA - Institut Polytechnique des Sciences Avancées

School Year 2025-2026

Contents

1 Introduction . . . 1
2 C code and time statistics of execution . . . 1
3 Scheduling analysis . . . 1
3.1 Schedulability . . . 1
3.2 Determining the best schedule . . . 2
4 Conclusion . . . 3

1 Introduction

This report presents the project for the final assignment of the real-time information system course. The objective is to study the behavior of a simple C code and find the best schedule to integrate this task in a task set.

2 C code and time statistics of execution

The C code is a simple multiplication of two big random numbers, between 1 and 1 000 000. The Python code compiles the C code and executes it 1000 times using the subprocess module and calculates the execution time.

The statistics of the execution time are the following :

- Min execution time: 5.95 milliseconds
- WCET execution time: 86.40 milliseconds
- Q1 execution time: 6.19 milliseconds
- Q2 execution time: 6.52 milliseconds
- Q3 execution time: 6.92 milliseconds

3 Scheduling analysis

3.1 Schedulability

To determine if the task is schedulable, we can use the utilization-based test for Rate Monotonic Scheduling (RMS). The utilization (U) of a task is calculated as:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

Where C is the worst-case execution time (WCET), T is the period of the task and n is the number of tasks. In our case if we use the WCET of 86.40 milliseconds and the given following task table :

Task	WCET (C)	Period (T)
τ_1	C_i	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

If we take the values of the table as ms, the utilization of the task set is calculated as follows:

$$U = \frac{86.40}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} = 8.64 + 0.3 + 0.1 + 0.1 + 0.05 + 0.05 + 0.0375 = 9.2875$$

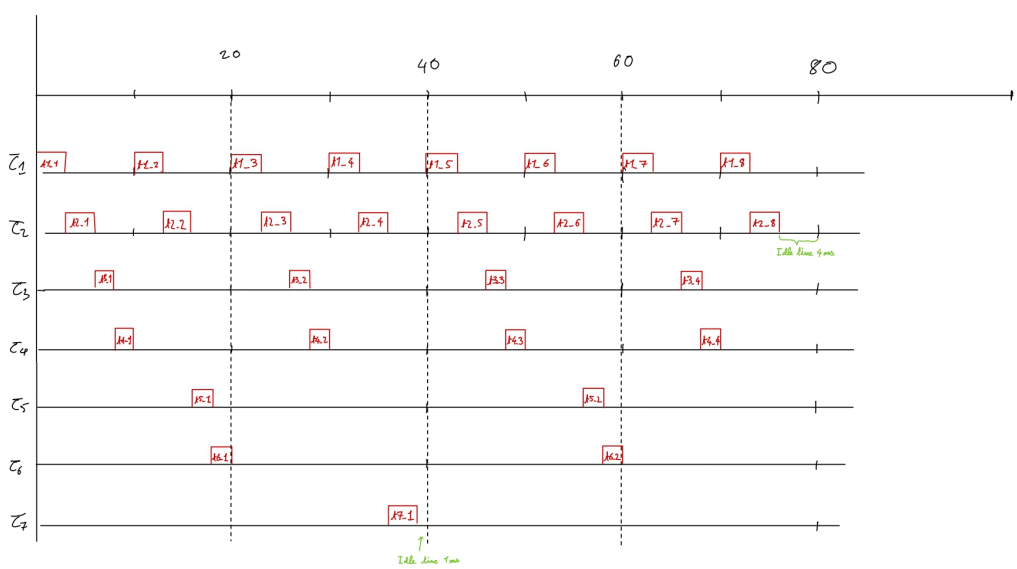
Since the utilization is greater than 1, the task set is not schedulable under RMS. Therefore for the study we will use an execution time of 3 milliseconds.
So the new utilization is calculated as follows:

$$U = \frac{3}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} = 0.3 + 0.3 + 0.1 + 0.1 + 0.05 + 0.05 + 0.0375 = 0.9375$$

Since the utilization is less than 1, the task set is schedulable under RMS.

3.2 Determining the best schedule

The hyperperiod of the task set is calculated as the least common multiple (LCM) of the periods of the tasks. The periods are 10, 20, 40, and 80 milliseconds. The LCM of these periods is 80 milliseconds. Therefore, the hyperperiod of the task set is 80 milliseconds. First we list all the possible schedules for the task set, then we calculate the response time for each task in each schedule and determine if the tasks meet their deadlines. The best schedule is the one that allows all tasks to meet their deadlines while minimizing the response time. After analyzing the schedules, we find that the best schedule is the one that allows all tasks to meet their deadlines while minimizing the response time. In this case, the best schedule is the following graph:



We obtain the following results:

Because Hyperperiode, C_i, and T_i are fixed parameters defined entirely by the task set, both expected idle time E_{total} and actual idle time I_{total} are fundamental constants of the system. A scheduling algorithm can neither create nor destroy computational work.

Therefore, minimizing the total waiting time across all jobs does not maximize the total processor idle time. Instead, an optimized schedule alters the distribution of the idle time. By minimizing delays and executing jobs as efficiently as possible, the algorithm consolidates the idle periods into larger, contiguous blocks (typically toward the end of the scheduling cycles), rather than fragmenting them into small, inefficient gaps between task executions.

4 Conclusion

In conclusion, this project provided a comprehensive analysis of a real-time system's behavior, from execution time profiling to task scheduling. Initially, we observed that the worst-case execution time (WCET) of the C program was too high (86.40 ms) to be integrated into the given task set under Rate Monotonic Scheduling (RMS), as the utilization factor exceeded 1. By assuming a bounded execution time of 3 ms, we successfully achieved a schedulable system with a valid utilization factor of 0.9375.

Furthermore, the scheduling analysis over the 80 ms hyperperiod highlighted a fundamental principle of real-time systems: while an optimized scheduling algorithm can minimize total waiting times and ensure all deadlines are met, it cannot alter the total processor idle time. Instead, an efficient schedule merely redistributes this idle time, consolidating it into larger blocks rather than fragmented delays. This project successfully demonstrated the importance of accurate execution time profiling and the practical implications of scheduling strategies in real-time environments.